



5

7

840 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Load balancing

د. روان قرعوني



RB Informatcs; 05/06/2025

محتوى مجاني غير مخصص للبيع التجاري

برمجة تفرعية

السلام عليكم ورحمة الله وبركاته

■ سنتكلم في هذه المحاضرة عن تنمة التجزئة وسنكمل ب Load Balancing

التجزئة

التجزئة لا تنفذ دائما بطريقة وحيدة يمكن اختيار عدة طرق بحيث نختار الطريقة حسب زمن التواصل او حجم الداتا

■ إذا كان لدينا مصفوفة مربعة هناك طريقتان للتوزيع بالمصفوفات :

1. إعطاء كل block لمعالج (نسميه مستوى block): نعتبره كتلة واحدة (خانة واحدة)
2. نوزع شريط كامل للمعالج (نسميه مستوى stripe)

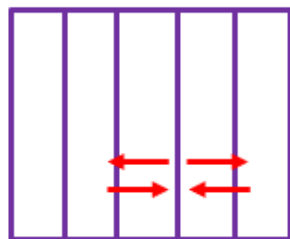
الزمن التفرعي: (زمن حساب وتواصل): $t_{computation} + t_{communication}$

بحيث زمن التواصل يكون : $t_{communication} = t_{startup} + (n * t_{data})$

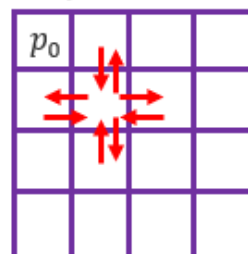
هو زمن نقل data واحدة مضروبا بحجم هذه الداتا + زمن تهيئة الارسال

اما زمن الحساب هو زمن المعالجة أي كل معالج يحتاج زمن لمعالجة البيانات
زمن التواصل بين هذه المعالجات = خلال عمليات الارسال والاستقبال

تحدثنا سابقا عن وجود طريقتين لتقسيم مصفوفة ,



stripe



block

1. الزمن بحالة ال block

بناء على الرسم السابق:

فرضاً: كل كتلة تتواصل مع ال 4 خانات المجاورة لها الزمن هو على شكل زائد
فكم عملية تواصل إرسال واستقبال يوجد؟

-يوجد لدينا 8 عمليات

عدد المعالجات لدينا $P = 16$

ف يكون زمن الارسال بحالة الكتلة (block)

$$\text{Block} = 8 * (t_{startup} + (\frac{n}{\sqrt{p}} * t_{data}))$$

بحيث $\frac{n}{\sqrt{p}}$ هو حجم الكتلة أي $\frac{n}{4}$

2. الزمن بحالة ال stripe

هنا يكون لدينا عمليتي تواصل ارسال واستقبال والخانة تتواصل مع الخانات المجاورة ك شريط لذلك يكون عدد العمليات 4

$$\text{Stripe} = 4 * (t_{startup} + (n * t_{data}))$$

ملاحظات

- في ال block حجم البيانات أقل لكن زمن التواصل أكبر
- في ال stripe حجم البيانات أكبر لكن زمن التواصل أقل
- ال block ليست دائماً احسن من ال stripe فهذا يعتمد على معاملات (زمن تواصل وحجم الداتا)

توزيع الحمل (Load Balancing)

نحيطكم علماً ان هذه المحاضرة مهمة جداً بالفحص

لدي عدة معالجات وأعرف مواصفات كل منهم وعندها عوضاً عن توزيع المهام بالتساوي على كل المعالجات أقوم بتوزيع متوازن بين المعالجات وأعطي الحمل المناسب لكل منها حتى انتهي بأقرب وقت ممكن , فهي توزيع المهام حسب قدرة المعالجات والمهام التي لدي.

يوجد لدينا نوعين للتوزيع :

1. static

في بداية البرنامج أفرض على كل معالج ماذا يستطيع أن يحمل على حسب قدرته

مشاكله: لا توجد دائماً القدرة على معرفة تفاصيل المسألة منذ البداية

استخدامه: قليل بمسائل محدودة

قد نضطر لعمل إعادة توزيع بعد المرور على الكود مرة ثانية

dynamic 2.

استخدامه اكثرسليباته: زمن تواصل اكبرايجابياته: load balancing مثاليويقسم الى مركزي وغير مركزي

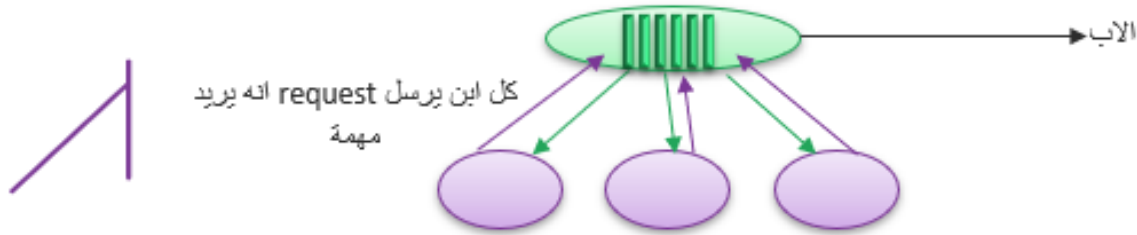
المركزي

يكون لدينا مهمة أب، تحوي مجموعة من الـ Tasks موجودة فيما يسمى بـ work pool بالإضافة إلى مجموعة أبناء

في هذه الطريقة يقوم كل ابن بإرسال request للمهمة الأب يُعلم الأب بأنه أنهى تنفيذ العمل المُوكل إليه فتقوم المهمة الأب بعملية send لنفس الابن تحوي على task جديد ليقوم بتنفيذه

بحيث يجزئ عمله على عدة Tasks وفيه مهام وفيه manager

الاب يحتوى على كل Tasks لعدم معرفته تفاصيل المسألة منذ البداية لذلك يوجد ضغط كبير على الاب



ملاحظة :

-يستطيع الابن ان يقوم بعملية submit أيضا أي أن يقوم بإضافة مهمة جديدة على الـ work pool

-تكون هذه المهمة الجديدة ظهرت أثناء تنفيذ عمل ما

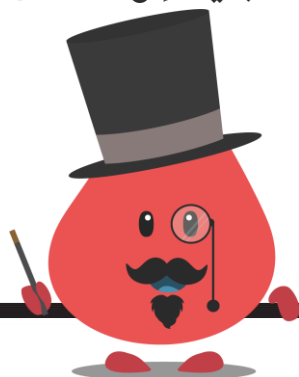
مثال :

لدينا برنامج يقوم بمعالجة صور وأثناء معالجة واحدة من الصور تبين أن هذه الصورة يجب ان تتم عليها:

■ عملية تعديل للألوان

■ علمية زيادة دقة

يقوم المعالج المُوكل إليه عملية معالجتها بإضافة الـ Tasks الجديدة إلى الـ work pool الموجود في المهمة الأب ليقوم بدوره بإرسالها إلى المعالجات المتاحة.



■ كيف نعلم ان التنفيذ انتهى في هذه الطريقة ؟

يجب ان يتحقق كلا الشرطين

■ ان يكون الـ work pool فارغ

■ ألا يوجد requests

تنتهي المهام عندما الكل ليس لديه مهام والكل قام بعمل request لـ task جديدة

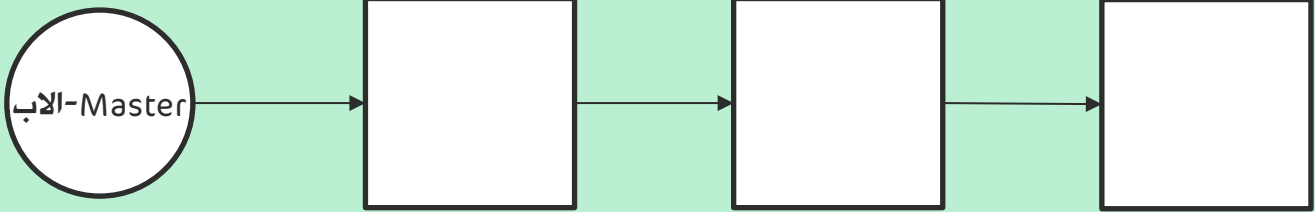
غير مركزي

■ يكون الـ work pool موزع على جميع المهام أي أن كل مهمة تحوي جزء منه

■ تقوم كل مهمة بتنفيذ ما تستطيع تنفيذه وتقوم بإرسال الـ Tasks لمهام أخرى في حال طلبها

ولتنفيذه يوجد 3 انواع

1. linear



الـ Tasks تترتب بشكل خطي فكل مهمة تتواصل مع المهمة التي قبلها لـ اخذ مهمة عندما تطلب التاسك مهمة ولا يتم الرد عليها وعندها تكون انتهت كل المهام

سليته زاد زمن التواصل

2 - fully_distributed

- كل مهمة تتواصل مع الكل أي لديها كل المهام

وتتواصل مع الكل عند بدء العمل

- المهمة الأولى تنبأ وتخبر الكل بذلك

- من الممكن ان يصبح تضارب conflict

- نوعه معقد



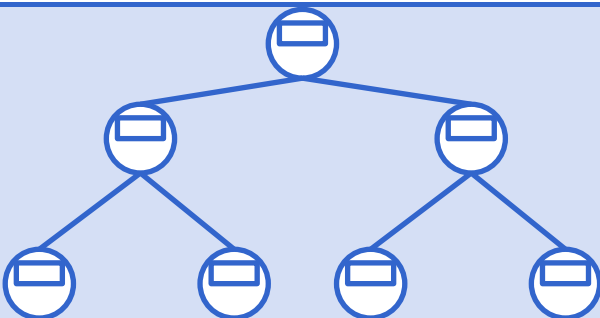
3 - tree

- كل ابن يرسل لوالده فيصبح لكل أب work pool خاص به

- هنا وفرنا عمليات التواصل

- فالتعقيد اصبح لوغاريتمي ($\log n$)

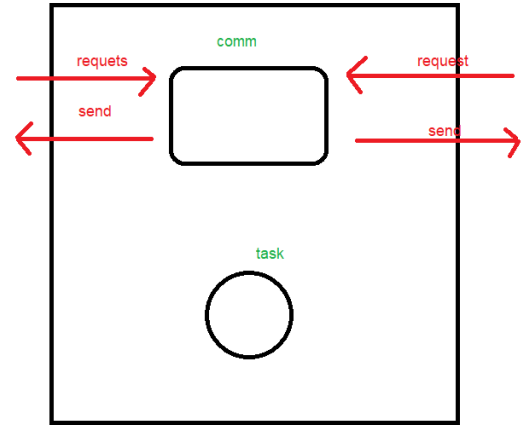
- لا يحتاج الجميع لانتظار دوره



تتم عملية التواصل بين المعالجات من خلال عمليتي send-request

ملاحظات:

- افضل شي من ناحية البنى هي البنية الشجرية
- ال historical data تفيد في معرفة أنواع المعالجات
- الخوارزميات التفرعية تتحسن مع مرور الزمن لانها ستعتمد على داتا سابقة

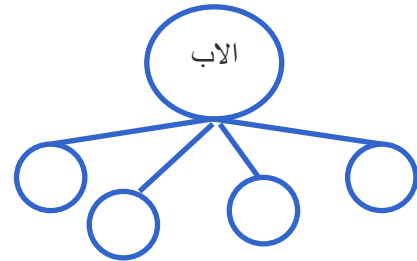


Termination Detection (اكتشاف الإنهاء)

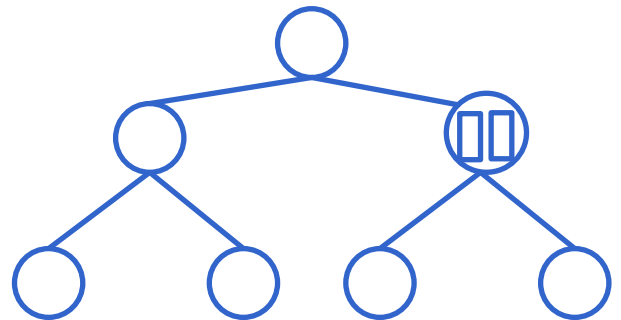
- متى اعلم ان كل المهام خلصت ؟ يعتمد ذلك على البنية التي اعلم عليها
- قد يتم استخدام token او علامة معينة

1- المركزي: يكون لدي اب مهمة يقود كامل العملية عندما تنتهي المهمة يرسل له , ف إما يرسل له تاسك جديدة او يعتبرها انتهت (ليس عنده تاسكات لها)

تعقيده n
يمر على كل الابناء



2- الشجري: كل مهمة ترسل لوالدها أيضا وهو لديه عمل بحد ذاته (لا ينتظر فحسب)
نعلم ان الفرع انتهى عندما أبناء الفرع ترسل رسالة بالانتهاء وهو ينتهي هكذا مع كل فرع وابناءه

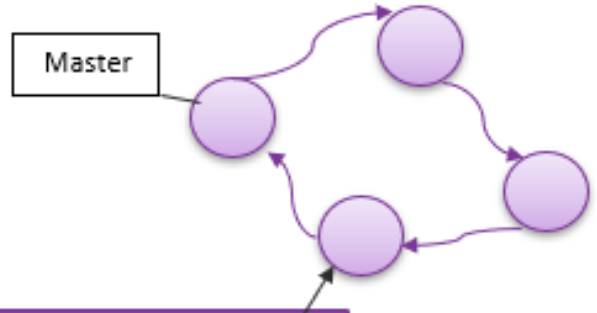
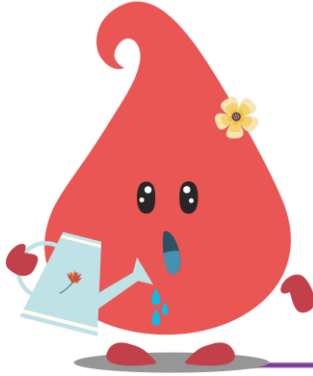


ملاحظات على البنية الشجرية

- بشكل مشابه للبيئة الخطية تقوم المهمة بإرسال token إلى المهمة الأب لها
- تقوم المهمة الأب من التحقق من حالتها فيما اذا كانت قد انتهت وفيما اذا وصلها token من جميع
المهام الأبناء لها

- تقوم هي بدورها بإرسال Token للمهمة التي قبلها في الشجرة
- وتستمر العملية حتى الوصول إلى المهمة الجذر في الشجرة والتحقق من أنها انتهت من التنفيذ

3-الحلقي Ring: ربط كل شيء بحلقة كل من ينتهي من العمل يخبر من بعده



هذا الابن لا يخبر انه انتهى حتى ينتهي عمله بشكل كامل (يرسل توكن)
لأنه بمجرد ان يرسل للاب فهذا يعني ان كل ما قبله انتهى

ملاحظات ختامية

منهاج المادة السنة القادمة سوف يتغير بنسبة 90%
لذلك ننصحكم بالاجتهاد والنجاح بالمادة هذا الفصل
المحاضرات تحتوي على الأفكار العامة والشرح عنها ولذلك لا بد من التوسع بدراسة مرجع المادة
والرجوع لدراسة القسم العملي من المادة

هنا نكون قد انتهينا من مقرر البرمجة التفرعية

نرجو من الله سبحانه وتعالى أن يتقبل عملنا وعملكم وتذكروا أن كل
الأحلام تتحقق يا أصدقاء

فريق البرمجة التفرعية

تصحيح أخطاء المحاضرات السابقة

المحاضرة رقم 3 صفحة 7

الخطأ

T1	T2	T3	T4
0	5	9	13
1	6	10	14
2	7	11	15
3	8	12	16



1	2	3	؟
5	6	7	8
9	10	10	12
12	15	10	16

T1	T2	T3	T4
1	2	3	4

التصحيح

T1	T2	T3	T4
0	5	9	13
1	6	10	14
2	7	11	15
3	8	12	16



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

المحاضرة رقم 3 صفحة 8

الخطأ

T1	T2	T3	T4
1	2	3	4

1	3	6	10
---	---	---	----

التصحيح

T1	T2	T3	T4
1	2	3	4

1	3	6	10
---	---	---	----